JEPA

V-JEPA 2

V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning

1. Introduction

문제

현대 AI의 큰 목표 중 하나는 **사람처럼 세상을 관찰하면서 현실의 구조와 변화 원리를 배우고, 처음 보는 상황에서도 적절하게 행동하는 것**

사람은 세상의 모든 상황을 직접 행동해보면서 배우는 것이 아님. 물건이 떨어지는 모습, 사람이 컵을 잡는 모습, 문이 열리는 모습 등을 관찰하면서 자연스럽게 다음과 같은 규칙을 배움.

- 물체를 밀면 이동함
- 손으로 물체를 잡으면 손과 물체가 함께 움직임
- 물체를 놓으면 중력 때문에 떨어질 수 있음
- 현재 상황을 보면 어느 정도 다음 상황을 예상할 수 있음 .... etc

이처럼 **현재 상태를 보고 미래 상태를 예측할 수 있는 내부 모델**을 월드 모델이라고 함.

월드 모델이 있으면 어떤 행동을 하면 원하는 결과가 나올지를 미리 생각하면서 계획할 수 있음!

기존 월드 모델의 문제

- 기존 로봇 월드 모델은 보통 다음과 같은 데이터를 필요로 했음.

  > 현재 상태 → 행동 → 다음 상태 → 성공 여부 또는 보상

  = 로봇을 실제로 움직이며 대량의 상호작용 데이터를 수집해야 하지만 실제 로봇 데이터는 비싸고 수집 속도도 느림

- 반대로 인터넷에는 엄청난 양의 영상이 존재함. (ex. 사람이 컵을 잡는 영상, 차가 움직이는 영상, 사람이 물건을 던지는 영상, 물체가 떨어지는 영상...)

  근데 이런 인터넷 영상에는 **정확한 행동 정보가 없음**.

  예를 들어 사람이 손을 움직이는 영상은 있지만 `이 시점에서 손을 x축으로 3cm 이동함` 같은 행동 명령은 기록되어 있지 않음.

논문 접근법

V-JEPA 2는 이 문제를 2단계 학습으로 해결

1단계: 인터넷 영상으로 세상 자체를 먼저 학습

목표: 영상에서 중요한 정보가 무엇이고, 장면이 시간에 따라 어떻게 변하는가를 학습하는 것

약 100만 시간 이상의 영상과 100만 장의 이미지를 이용해 V-JEPA 2를 자기지도학습

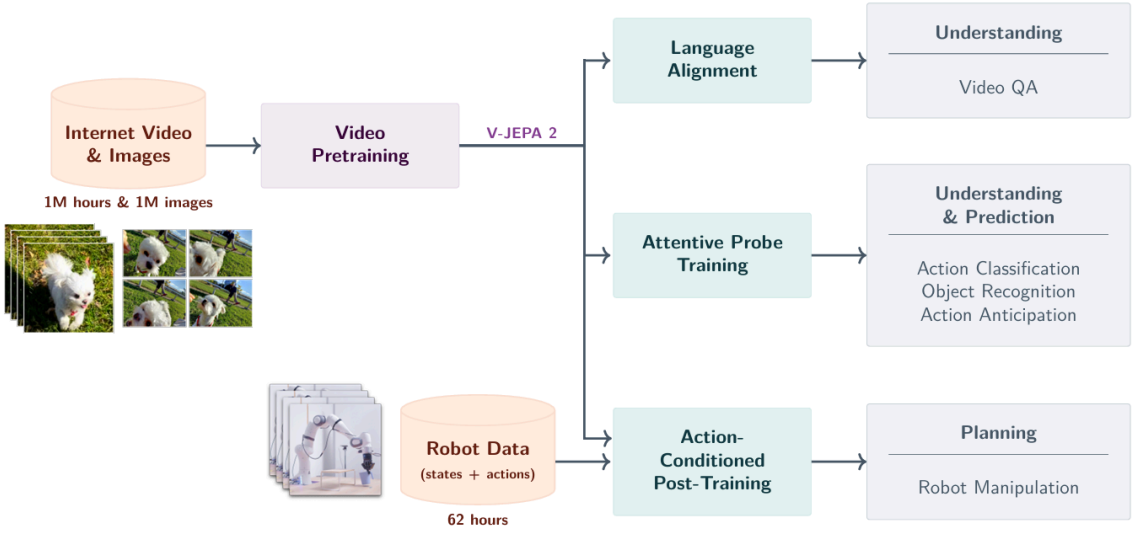
이 단계에서는 행동 정보가 전혀 필요하지 않음.

2단계: 소량의 로봇 데이터로 행동의 효과를 추가 학습

V-JEPA 2의 영상 표현은 그대로 두고, 62시간 미만의 로봇 영상으로 `이 상태에서 이 행동을 하면 다음 상태가 어떻게 변하는가` 를 학습 → 이 모델이 V-JEPA 2-AC(Action-Conditioned)

최종적으로 논문은 한 기반 모델을 세 가지 방향으로 활용함.

기능	무엇을 하는가
Understanding	영상 속 행동, 물체, 상황 이해
Prediction	이후에 일어날 행동과 상태 예측
Planning	목표 상태에 도달하기 위한 로봇 행동 계획



모델 참고

모델	역할
V-JEPA 2	대규모 인터넷 영상으로 학습된 기본 비디오 모델
V-JEPA 2-AC	V-JEPA 2 위에 행동 조건부 예측기를 붙인 로봇용 월드 모델
V-JEPA 2 + LLM	V-JEPA 2를 시각 인코더로 사용하고 언어모델과 연결한 비디오 질의응답 모델

2. 핵심 개념

Representation

컴퓨터에게 영상은 결국 픽셀 숫자 배열임

하지만 모델이 영상을 이해하려면 모든 픽셀을 그대로 기억하는 것보다 중요한 특징을 숫자로 압축하는 것이 유리함.

예를 들어 사람이 컵을 집는 영상이라면 모델 내부에서는 사람, 손, 컵, 손과 컵이 가까움, 컵을 잡고 있음, 오른쪽으로 이동 중 같은 정보가 숫자 벡터 형태로 표현될 수 있음.

→ 이런 내부 표현을 **Representation or Embedding**이라고 하고

→ 이러한 표현들이 존재하는 공간을 **Representation Space 또는 Latent Space**라고 함.

JEPA의 핵심 아이디어

기존 영상 생성 방식에서는 현재영상 → 미래영상픽셀을 예측함

예를 들어 다음 장면을 만들려면 손과 컵뿐 아니라 배경의 벽, 나뭇잎, 그림자, 조명, 물체 질감까지 모두 만들어야 함.

하지만 로봇이 행동을 결정할 때 이런 세부사항이 반드시 필요한 것은 아니다..

V-JEPA는 대신 현재 Representation → 미래 Representation을 예측함.

= 픽셀을 생성하지 않고 의미 있는 특징을 예측함.

예를 들어 정확한 컵 표면 질감은 몰라도 `컵이 오른쪽으로 이동함` 이라는 구조적인 정보만 잘 표현하면 됨

논문에서는 생성 모델이 정확한 풀잎이나 나뭇잎 위치처럼 예측하기 어려운 픽셀 정보까지 맞히려 하는 반면, JEPA는 물체의 궤적처럼 **예측 가능한 구조적 정보에 집중**한다고 설명함.

---

3. V-JEPA 2: 자기지도 비디오 사전학습

3.1. 기본 학습 방식

V-JEPA 2는 영상 일부를 가리고 가려진 부분의 Representation이 무엇이었을지를 맞히면서 학습함.

중요한 점은 **가려진 픽셀을 복원하는 것이 아니라 가려진 부분의 특징 표현을 예측한다는 것**

구조는 크게 세 부분

구성	역할
Encoder	보이는 영상에서 특징 추출
Predictor	보이는 특징을 바탕으로 가려진 부분 예측
Target Encoder	원본 영상으로 정답 Representation 생성

Mask-Denoising Objective

$\min_{\theta,\phi,\Delta_y} \|P_\phi(\Delta_y, E_\theta(x)) - \mathrm{sg}(E_{\bar{\theta}}(y))\|_1$

- y: 가리지 않은 원본 영상
- x: 원본 영상의 일부 patch를 제거한 영상
- E_θ: Encoder
- θ: Encoder의 파라미터
- P_ϕ: Predictor

- ϕ: Predictor의 파라미터
- Δ_y: 가려진 위치를 알려주는 학습 가능한 mask token
- $E_{\bar{\theta}}(y)$: 원본 영상을 Target Encoder에 넣어 얻은 정답 Representation
- sg: Stop-Gradient
- $\|\cdot\|_1$: L1 거리

$P_\phi(\Delta_y, E_\theta(x))$: 가려진 영상을 보고 모델이 예상한 Representation
$E_{\bar{\theta}}(y)$: 원본 영상 전체를 봤을 때 얻은 실제 Representation

⇒ 전체 수식은 **예측한 Representation과 실제 Representation의 차이를 최소화**하는 것

L1 Loss

예를 들어 실제 Representation이 [1,3,5], 예측값이 [2,2,7]이라면 → |2-1|+|2-3|+|7-5| =4

차이가 작아질수록 모델의 예측이 실제 Representation과 비슷해짐.

Loss는 전체 영상이 아니라 **가려진 patch의 prediction에 대해서만 계산함**.

Stop-Gradient가 왜 필요한가?

Target Encoder까지 함께 gradient로 업데이트하면 아주 쉬운 편법이 가능함.

모든 영상을 [0,0,0,...]같은 Representation으로 만들어버리면 prediction과 target이 항상 같아져 Loss가 0이 될 수 있음.

하지만 아무 의미도 배우지 못함.

⇒ 이를 **Representation Collapse**라고 함.

그래서 정답을 만드는 Target Encoder 쪽에는 gradient를 전달하지 않음.

EMA Encoder

Target Encoder는 완전히 고정된 건 아니고 Main Encoder를 천천히 따라감.

개념적으로 $\bar{\theta}_t = m\bar{\theta}_{t-1} + (1-m)\theta_t$ 형태의 EMA(Exponential Moving Average)를 사용

= 최신 Encoder를 바로 복사하지 않고 조금씩 반영함.

→ Target Encoder가 상대적으로 안정적인 정답 생성기 역할을 하게 됨

영상은 어떻게 Transformer에 들어가는가?

영상을 그대로 넣지 않고 **Tubelet**이라는 작은 블록으로 자름.

V-JEPA 2의 tubelet 크기는 2*16*16 이고 순서는 T * H * W임

- 시간축: 2 frame
- 높이: 16 pixel
- 너비: 16 pixel

단위로 영상이 잘려 하나의 token이 됨.

이미지 ViT가 이미지를 16×16 patch로 자르는 것에 **시간축 2 frame이 추가된 형태**

3D-RoPE

Transformer는 token만 보면 그 token이 영상의 어느 위치에서 왔는지 알 수 없음. → 그래서 위치 정보를 추가함.

기존 V-JEPA는 absolute sin-cos positional embedding을 사용했지만 V-JEPA 2는 **3D Rotary Position Embedding**을 사용함.

Feature dimension을 대략 세 부분으로 나눠 각각

- 시간 T
- 높이 H
- 너비 W

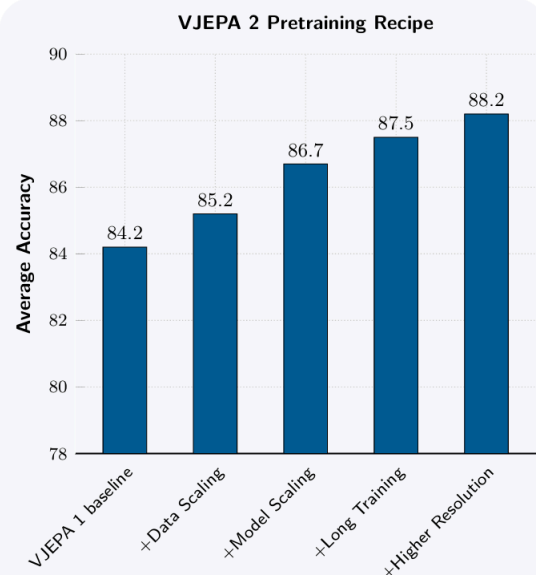
위치 정보를 넣음.

저자들은 3D-RoPE가 특히 큰 ViT-g 모델을 학습할 때 안정성을 높였다고 보고함

3.2. V-JEPA 2의 스케일링

기존 V-JEPA에서 다음 네 가지를 크게 확장함.

요소	기존	V-JEPA 2
데이터	약 2M videos	약 22M samples
모델	ViT-L, 약 300M	ViT-g, 약 1B
학습 길이	약 90K iterations	최대 252K
영상 크기	짧은 16-frame 중심	최대 64 frame 고해상도



V-JEPA 1 baseline 을 기본으로 잡고 아래 조건을 하나씩 추가

- Data Scaling
- Model Scaling
- Long Training
- Higher Resolution

= 단순히 모델만 크게 만든 것이 아니라 **데이터, 모델 크기, 학습량, 입력 해상도를 함께 확장함**.

3.3. Pretraining Dataset

최종 사전학습 데이터셋: **VideoMix22M, VM22M**

데이터	Sample 수	특징	총 영상 시간	Sampling Weight
SSv2	168K	사람 중심 동작 영상	168h	0.056
Kinetics	733K	일반 행동 영상	614h	0.188
HowTo100M	1.1M	유튜브 튜토리얼	134K h	0.318
YT-Temporal-1B	19M	일반 유튜브	1.6M h	0.188
ImageNet	1M	정지 이미지	해당 없음	0.250

ImageNet은 영상이 아닌데?

ImageNet 이미지를 시간축으로 반복함.

예를 들어 한 이미지를 16번 복사해서 동일한 frame이 16번 이어지는 영상처럼 처리함.

→ 이를 통해 motion뿐 아니라 **물체와 장면의 appearance 정보도 강화**

3.4. YT1B Data Curation

YT-Temporal-1B는 인터넷에서 대규모로 수집된 데이터라 노이즈가 많음.

예를 들어 만화 영상, Clipart, 품질이 매우 낮은 영상, 실제 행동 이해에 도움이 적은 장면 등이 섞일 수 있음.

그래서 별도의 데이터 정제 과정 수행

방식

1. PySceneDetect를 이용해 scene transition 기준으로 영상을 분리
2. 4초 미만 scene 제거
3. 약 316M scene 확보
4. DINOv2로 각 scene의 embedding 계산
5. 약 1.5M cluster 생성
6. Kinetics, SSv2, COIN, EpicKitchen과 비슷한 cluster만 선택

Cluster Sampling Weight

cluster c의 sampling weight : $w_c = \sum_{d=1}^{D} w_d \frac{N_{d,c}}{N_d}$

- c: cluster
- d: target dataset
- w_d: target dataset의 weight
- $N_{d,c}$: dataset d에서 cluster \(c\)에 속하는 sample 수
- N_d: dataset d 전체 sample 수

= **Kinetics나 SSv2처럼 우리가 원하는 고품질 데이터와 비슷한 cluster를 더 많이 뽑자**는 의미

Curated YT1B를 사용하면 ViT-L 실험에서 uncurated YT1B 대비 평균 성능이 약 1.4 point 증가함.

3.5. Model Scaling

Encoder 구조:

모델	Parameter	Width	Layer	Head	MLP
ViT-L	300M	1024	24	16	4096
ViT-H	600M	1280	32	16	5120
ViT-g	1B	1408	40	22	6144

Pretraining predictor는 모든 모델에서 비교적 작은 **ViT-s 약 22M parameter**를 사용

Encoder를 300M에서 1B로 늘렸을 때 downstream 평균 성능이 약 1.5에서 1.7 point 정도 향상됨. 측정 조건에 따라 수치가 조금 다르게 보고되지만 전반적으로 모델 크기를 키울수록 성능이 향상된다

3.6. Training Schedule

기존 cosine schedule 대신 **warmup** → **constant** → **cooldown** 구조 사용

- 12K iteration: learning rate warmup
- 228K iteration: constant learning rate
- 마지막 12K: cooldown

⇒ constant phase의 여러 checkpoint에서 cooldown을 시작할 수 있어 장기 학습을 효율적으로 실험할 수 있음

3.7. Progressive Resolution Training

문제: ViT-g를 처음부터 64frames, 384 * 384 입력으로 학습하면 계산량이 매우 큼

그래서 처음에는 16 frames, 256×256으로 대부분 학습하고 마지막 cooldown에서만 64 frames, 384×384 등으로 증가시킴.

영상 길이를 늘리면?

Pretraining 때 16 frame 대신 64 frame을 사용하면 evaluation은 동일하게 16 frame으로 해도 평균 성능이 약 0.7 point 증가

즉 학습 과정에서 더 긴 시간 정보를 경험하는 것 자체가 Representation 품질에 도움이 됨.

** 128, 256 frame까지 늘렸을 때는 해당 understanding task에서 64 frame 이상 추가적인 이득은 나타나지 않음.

---

4. V-JEPA 2-AC: 행동 조건부 월드 모델

V-JEPA 2는 영상 자체의 변화는 학습했지만 내가 어떤 행동을 했을 때 세상이 어떻게 달라지는지는 아직 모름.

인터넷 영상에는 robot action이 없었기 때문

그래서 사전학습된 V-JEPA 2 Encoder를 **freeze**하고 그 위에 새로운 Action-Conditioned Predictor를 학습

⇒ V-JEPA 2-AC

4.1. Robot Dataset

DROID dataset 사용: Franka Emika Panda 7-DoF robot arm이 다양한 물체를 조작하는 영상으로 구성됨.

** **Unlabeled Robot Video**는 action 정보가 없다는 뜻이 아님

- 사용하지 않는 정보:
  - 작업 종류
  - Reward
  - 성공/실패 label
- 사용하는 정보:
  - RGB 영상
  - End-effector 상태

4.2. 입력 데이터

4초 clip을 사용하고 4fps로 sampling
4 sec * 4fps = 16frames

각 frame: $(x_k)_{k\in[16]}$

Robot end-effector state: $s_k \in \mathbb{R}^7$

구성 $s_k = [x, y, z, r_1, r_2, r_3, g]$

- 앞 3개: Cartesian position
- 다음 3개: Euler angle orientation
- 마지막 1개: gripper state

Action

두 frame 사이 end-effector 상태 변화로 action 생성.

개념적으로 $a_k = s_{k+1} - s_k$이며 $a_k \in \mathbb{R}^7$임.

= 별도의 사람이 붙인 action label이 아니라 **연속된 robot state의 차이에서 행동을 계산**

Video Representation

각 frame을 frozen V-JEPA 2 Encoder에 넣음 : $z_k = E(x_k)$

$z_k \in \mathbb{R}^{H\times W\times D}$

ViT-g를 사용할 경우 실제 feature map 크기는 16*16*1408

4.3. Teacher-Forcing Loss

현재 상태와 action을 보고 **다음 frame Representation을 예측:**
$L_{\text{teacher-forcing}}(\phi) = \frac{1}{T}\sum_{k=1}^{T} \|\hat{z}_{k+1} - z_{k+1}\|_1$
→ $L_{\text{teacher-forcing}}(\phi) \frac{1}{T}\sum_{k=1}^{T} \left\| P_\phi\left((a_t, s_t, E(x_t))_{t\leq k}\right) E(x_{k+1}) \right\|_1$ (T=15)

= 현재 영상 + 현재 robot 자세 + 현재 행동 → 다음 장면의 Representation을 맞혀라

Teacher Forcing이란?

예를 들어 z_1 → $\hat{z}_2$를 예측했다고 하자.

다음 prediction을 할 때 모델이 예측한 $\hat{z}_2$를 넣는 것이 아니라 실제 정답 z_2를 넣음.

즉 학습에서는 $z_1 \rightarrow \hat{z}_2$, $z_2 \rightarrow \hat{z}_3$처럼 항상 실제 상태를 입력함.

장점은 학습이 안정적이라는 것.

하지만 실제 실행에서는 미래의 정답 z_2를 알 수 없고 자기 prediction을 다시 입력해야 함.

그래서 작은 오차가 계속 쌓일 수 있음.

4.4. Rollout Loss

이를 보완하기 위해 자기 예측을 다시 입력으로 사용하는 학습도 수행함.
$L_{\text{rollout}}(\phi) = \|P_\phi(a_{1:T}; s_1, z_1) - z_{T+1}\|_1$

예: $z_1 \rightarrow \hat{z}_2 \rightarrow \hat{z}_3$ 처럼 자신의 prediction을 이용해 여러 step을 전개한 뒤 마지막 prediction을 실제 미래 Representation과 비교함.

실제로는 T=2 를 사용

4.5. 최종 Loss

두 Loss를 합침.

⇒ $L(\phi) = L_{\text{teacher-forcing}}(\phi) + L_{\text{rollout}}(\phi)$\tag{4}

즉 한 step prediction도 잘하고 자기 prediction을 반복해도 error가 덜 쌓이게 하자는 목적

4.6. V-JEPA 2-AC Predictor 구조

약 **300M parameter Transformer** 사용.

- 24 layers
- 16 heads
- hidden dimension 1024
- GELU activation

Action, robot pose, visual feature는 각각 별도의 affine transformation을 거쳐 동일한 hidden dimension으로 변환됨.

Attention은 **Block-Causal Attention**을 사용함.

현재 \(t\) 시점은 $t,\ t-1,\ t-2, \ldots$ 정보를 볼 수 있지만 미래 시점의 정보는 볼 수 없음.

즉 미래를 미리 보고 답하는 것을 막음.

5. Understanding: Probe-based Classification

V-JEPA 2가 학습한 Representation 안에 실제 영상 이해에 필요한 정보가 얼마나 잘 들어 있는지 평가하는 실험임. Encoder는 **freeze**하고 그 위에 작은 분류기인 Probe만 학습함. 즉 Probe가 좋은 성능을 낸다면 V-JEPA 2가 사전학습 과정에서 이미 유용한 영상 정보를 Representation에 담았다고 볼 수 있음. jepa

평가 task는 크게 두 종류로 나눔.

- **Motion Understanding:** 시간에 따른 움직임을 봐야 정답을 알 수 있는 task
  - Something-Something v2
  - Diving-48
  - Jester
- **Appearance Understanding:** 한 장의 frame만으로도 상당한 정보를 얻을 수 있는 task
  - Kinetics-400
  - COIN
  - ImageNet

Attentive Probe

단순한 Linear Classifier가 아니라 **4-layer Transformer Probe**를 사용함. 처음 3개 block은 Self-Attention, 마지막 block은 learnable query token을 이용한 Cross-Attention으로 구성됨.

즉 query token이 V-JEPA 2의 많은 visual token 중 분류에 필요한 정보를 모아서 최종 label을 예측함.

따라서 이 실험은 정확히 말하면 Linear Probe 실험은 아님.

주요 결과

V-JEPA 2 ViT-g384는 평균 88.2를 기록함.

Task	성능
SSv2	77.3
Diving-48	90.2
Jester	97.8
K400	87.3
COIN	91.1
ImageNet	85.1

특히 SSv2처럼 **시간적 움직임을 이해해야 하는 task에서 강한 성능**을 보임.

반면 ImageNet처럼 appearance가 중요한 task에서는 SigLIP2나 다른 image encoder가 더 높은 경우도 있음.

= V-JEPA 2의 강점은 모든 이미지 task에서 무조건 우수한 것이 아니라 **영상의 시간적 변화와 motion 이해에 강함**

추가 실험에서는 1-layer Probe보다 4-layer Probe를 사용할 때 평균 성능이 약 1 point 정도 상승함.

따라서 결과에는 V-JEPA 2 Representation뿐 아니라 Probe 구조의 영향도 일부 존재함.

또 Diving-48과 Jester에서는 마지막 Encoder layer 하나만 사용하는 것보다 여러 intermediate layer의 feature를 함께 사용하는 것이 더 좋았음.

---

6. Prediction: Probe-based Action Anticipation

현재까지의 영상을 보고 **1초 뒤 사람이 어떤 행동을 할지 예측**하는 실험

Dataset은 Epic-Kitchens-100(EK100)을 사용

- 약 100시간의 주방 영상
- 97개 Verb
- 300개 Noun
- 총 3,568개의 Action

예를 들어

- Verb = open
- Noun = fridge
- Action = open fridge

예측 방식

현재 영상은 V-JEPA 2 Encoder에 넣어 현재 Representation을 얻음. 이후 V-JEPA 2 Predictor가 **1초 미래의 Representation을 먼저 예측**함.

이 두 정보를 함께 Probe에 입력함.

현재 Encoder Feature + 미래 Predictor Feature → Attentive Probe → Verb / Noun / Action 예측

Probe에는 각각 Verb, Noun, Action을 담당하는 3개의 query token이 존재함.

평가 지표

미래 행동은 하나로 확정되어 있지 않기 때문에 Accuracy 대신 **Recall@5**를 사용함.

즉 모델이 예측한 상위 5개 행동 중 실제 행동이 포함되어 있으면 성공으로 판단함.

주요 결과

Model	Action Recall@5
PlausiVL	27.6
V-JEPA 2 ViT-L	32.7
V-JEPA 2 ViT-H	36.5
V-JEPA 2 ViT-g	38.0
V-JEPA 2 ViT-g384	39.7

기존 PlausiVL의 27.6에서 39.7로 증가함.

상대적 향상률은 일반 텍스트로 계산하면: (39.7 - 27.6) / 27.6 ≈ 44% 정도

Predictor가 실제로 얼마나 중요한가?

입력	Verb	Noun	Action
Encoder만	61.3	57.0	39.1
Predictor만	48.7	34.7	20.2
Encoder + Predictor	63.6	57.1	39.7

핵심은 **Encoder만 사용해도 Action Recall@5가 39.1이라는 점**임. Predictor를 추가하면 39.7로 약간 올라감.

즉 높은 성능의 대부분은 미래를 정확하게 시뮬레이션해서 나온 것이 아니라 **현재 상황을 잘 이해하는 강한 Representation 덕분**임. 논문 역시 EK100이 순수한 forecasting보다 semantic understanding에 크게 의존한다고 해석함.

Longer-term Prediction

미래를 1초, 2초, 4초, 10초까지 멀리 예측하게 하면 성능이 크게 감소함.

미래가 멀어질수록 사람이 할 수 있는 행동의 경우의 수가 많아지고 예측이 불확실해지기 때문임. 따라서 V-JEPA 2가 장기 미래까지 안정적으로 예측한다고 보기는 어려움.

---

7. Understanding: Video Question Answering

V-JEPA 2 자체는 언어모델이 아니므로 질문에 문장으로 답할 수 없음. 따라서 V-JEPA 2를 **Visual Encoder**로 사용하고 LLM을 연결함.

전체 구조: Video → V-JEPA 2 → Visual Embedding → Projector → LLM → Answer

Projector는 V-JEPA 2가 만든 visual feature를 LLM이 이해할 수 있는 차원으로 변환하는 역할을 함.

이 실험에서 중요한 점은 **V-JEPA 2의 사전학습에는 언어 데이터가 사용되지 않았다는 것**임. 즉 caption이나 image-text pair 없이 영상만 보고 학습했는데도, 이후 LLM과 연결했을 때 Video QA에서 좋은 성능을 보였음.

Visual Instruction Tuning

단계적으로 LLM과 visual representation을 정렬함.

- Stage 1: Image Captioning으로 Projector 정렬
- Stage 1.5: 고품질 Image Captioning으로 Full Model 학습
- Stage 2: 대규모 Image Question Answering
- Stage 3: Video Captioning + Video Question Answering

뒤 단계로 갈수록 LLM이 V-JEPA 2의 visual token을 더 잘 이해하도록 학습함.

7.1. Comparing with Image Encoders

공정한 비교를 위해 LLM과 학습 데이터는 동일하게 유지하고 Vision Encoder만 변경함.

- LLM: Qwen2-7B-Instruct
- Alignment Data: 18M image/video-text pairs
- Vision Encoder: Frozen

Encoder	평균
DINOv2	45.7
SigLIP2	48.1
Perception Encoder	49.1
V-JEPA 2	52.3

V-JEPA 2는 특히 MVP, TemporalBench, TVBench처럼 **시간적인 정보를 이해해야 하는 Video QA에서 강한 성능**을 보임. 반면 PerceptionTest에서는 일부 image encoder보다 낮은 성능도 나타남.

7.2. Scaling Vision Encoder Size and Input Resolution

모델 크기와 입력 해상도를 늘릴수록 성능이 점진적으로 증가함.

Model	평균
ViT-L256	51.7
ViT-H256	52.0
ViT-g256	52.3
ViT-g384	54.0
ViT-g512	54.4

즉 Video QA에서도 **더 큰 Encoder와 더 높은 resolution이 도움이 됨**.

7.3. Improving the State-of-the-art by Scaling Data

Alignment dataset을 18M에서 **88.5M**으로 확대하고 LLM도 Llama 3.1 8B로 변경함.

최종 V-JEPA 2 + Llama 3.1 8B의 평균 성능은 59.5임.

Benchmark	성능
PerceptionTest	84.0
MVP	44.5
TempCompass	76.9
TemporalBench	36.7
TOMATO	40.3
TVBench	60.6
MVBench	73.5

PerceptionTest, MVP, TempCompass, TemporalBench, TOMATO에서는 강한 결과를 보였지만 TVBench와 MVBench에서는 PerceptionLM이 더 높은 성능을 기록함. 즉 모든 Video QA benchmark에서 항상 최고인 것은 아님.

긴 영상에서는 왜 유리한가?

Figure 19에서는 입력 frame 수를 계속 늘려 V-JEPA 2와 DINOv2를 비교함.

- V-JEPA 2: frame 수가 증가할수록 성능 증가
- DINOv2: frame 수를 늘려도 성능이 정체되거나 감소

DINOv2는 기본적으로 image encoder이므로 각 frame의 appearance 정보에는 강하지만, V-JEPA 2는 처음부터 영상의 temporal structure를 학습했기 때문에 **긴 영상의 시간 흐름을 활용하는 데 더 유리한 가능성**을 보여줌.

---

8. Related Work

World Models and Planning

기존에도 현재 상태 + 행동 → 미래 상태를 예측하는 World Model 연구는 존재했음.

다만 기존 real-world robot 연구는 특정 환경에서 많은 robot interaction data를 직접 수집해야 하는 경우가 많았음.

V-JEPA 2의 차이는 대규모 인터넷 영상 → 일반적인 visual representation 학습 → 적은 robot data로 action dynamics 추가 학습이라는 구조

Vision-Language-Action Models for Robotic Control

RT-2, Octo, OpenVLA 같은 모델은 일반적으로 현재 상황 → 행동을 직접 학습함.

즉 expert demonstration을 통해 **어떤 상황에서 어떤 행동을 해야 하는지 policy를 학습**하는 방식임.

반면 V-JEPA 2-AC는 현재 상황 + 행동 → 미래 상황을 학습함.

그 후 여러 행동 후보를 World Model 안에서 시험하고 목표와 가장 가까워지는 행동을 선택함.

따라서 차이는 다음과 같음.

- VLA: Policy Learning
- V-JEPA 2-AC: World Model + Planning

Vision Foundation Models

DINOv2, VideoMAE, InternVideo, VideoPrism 등 기존 Foundation Model도 대규모 visual pretraining을 수행함.

하지만 대부분 Classification, Recognition, Video QA 같은 **Understanding Task**를 중심으로 평가했음.

V-JEPA 2는 여기서 한 단계 더 나아가 학습한 Representation을 실제 physical world에서 Prediction과 Planning까지 사용할 수 있는가를 검증했다는 점에서 차이가 있음.

---

9. Conclusion

V-JEPA 2의 핵심은 **대규모 영상 관찰만으로 학습한 Representation을 Understanding, Prediction, Planning에 공통으로 사용할 수 있다는 가능성을 보여준 것**임.

전체 흐름은 다음처럼 볼 수 있음.

인터넷 영상 → V-JEPA 2 → 일반적인 Video Representation 학습

이 Representation을 그대로 사용하면

- Action Classification
- Object Recognition

- Action Anticipation
- Video Question Answering

등에 활용할 수 있음.

여기에 소량의 robot interaction data를 추가하면 V-JEPA 2-AC가 됨.

현재 상태 + Action → 미래 Representation을 예측하게 되고, 여러 Action 후보의 미래를 미리 예측해 Goal과 가장 가까운 행동을 선택함.

논문의 핵심 차별점은 끝까지 **미래의 Pixel을 직접 생성하지 않는다는 것**임.

기존 생성형 World Model: 현재 상태 → 미래 Pixel

V-JEPA 계열: 현재 Representation → 미래 Representation

즉 배경이나 질감처럼 행동에 중요하지 않을 수 있는 세부 Pixel까지 생성하는 데 계산량을 쓰기보다, **물체, 움직임, 위치 관계처럼 현실을 이해하고 행동하는 데 필요한 정보에 집중해서 미래를 예측하는 방식**

향후 과제로는 세 가지를 제시함.

- **Long-Horizon Prediction:** 현재는 미래가 길어질수록 prediction error가 커지므로 여러 시간 scale을 다루는 hierarchical world model 필요
- **Language Goal:** 현재 robot planning은 Goal Image가 필요하므로 자연어 instruction을 latent space와 연결할 필요
- **Model Scaling:** 현재 최대 약 1B parameter이므로 더 큰 model에서도 scaling 효과가 이어지는지 확인 필요